

《DIZK：分布式零知识证明系统》逐页演讲稿

对应文件： 分布式零知识证明(1).pptx

幻灯片页数：18 页

建议汇报时长：约 15~18 分钟

使用方法：每页先看“本页重点”，再按“演讲稿”展开。括号中的内容是讲解提示，不必逐字念出。

第 1 页：封面

本页重点： 介绍论文、研究背景和核心问题。

演讲稿：

各位老师、同学，大家好。今天我汇报的论文是 DIZK，全称是分布式零知识证明系统。

这篇论文关注的是 zkSNARK，也就是零知识简洁非交互式知识论证。在这类系统中，验证一份证明通常很快，但生成证明需要处理规模庞大的多项式和群运算，单机很容易因为内存不足而失败。而DIZK正是解决上述问题

过渡句： 在介绍 DIZK 之前，先简单回顾一下零知识证明解决的基本问题。

第 2 页：零知识证明与 zkSNARK

本页重点： 解释零知识、完备性、可靠性和简洁性。

演讲稿：

零知识证明的价值最初体现在阿里巴巴与四十大盗的故事里，四十大盗抓住了阿里巴巴，但是他们想获得宝藏需要通过关卡，他们自己不会，只能借助阿里巴巴，但是阿里巴巴就想呀：如果我帮助大盗通过关卡拿到宝藏，自己会因为失去利用价值而被杀掉，但是如果什么都不说，大盗可能就以为自己本来就一无所知而被杀掉，所以想让大盗知道我真的能解开关卡，但又不打算向大盗透露是如何解密的

用一句话概括：我希望让别人相信我的陈述是真的，但不需要把支撑这个陈述的秘密交出来。

一个完整的零知识证明系统通常强调几项性质。简洁性：验证速度极快。完备性：如果陈述为真、证明者也确实知道证人，那么诚实生成的证明应当通过验证。可靠性：如果陈述是假的，攻

击者几乎不可能伪造一份可通过验证的证明。零知识性保证验证者不会从证明中得到关于秘密证人的额外信息。

需要说明的是，这些是 zkSNARK 的通用性质，并不是 DIZK 新提出的概念。

过渡句：zkSNARK 的验证虽然很轻量，但三个阶段的成本并不对称。

第 3 页：zkSNARK 的三个阶段

本页重点：验证轻量，Setup 和 Prover 随电路规模增长。

演讲稿：

zkSNARK 通常包含 Setup、Prover 和 Verifier 三个阶段。

Setup 首先输入部分：R1CS（一阶约束系统）--zksnark无法直接看懂计算机程序，需要将计算任务转化为一种底层的数学矩阵，通常会有很多等式，就需要进行 QAP（二次程序转换）--将约束方程进行压缩打包方便验证者证明。

针对一个固定的计算关系或约束系统运行，生成证明密钥 (pk) 和验证密钥 (vk)。对于同一个计算关系，这一步只需要执行一次，之后可以支持多次证明。

Prover 接收证明密钥、公开输入和秘密证人，生成证明 (π)。这一阶段包含大量多项式运算和群上的多标量乘，是整个系统中计算和内存开销最大的部分。

Verifier 接收验证密钥、公开输入和证明，输出接受或拒绝。它只需要执行少量配对运算。论文给出的验证成本大约是 2 毫秒，再加上与公开输入数量有关的一小部分时间。

因此，zkSNARK 存在明显的不对称：证明只有 128 字节，验证非常快，但 Setup 和 Prover 的内存消耗与逻辑门电路成准线性关系。DIZK 要解决的正是后两者的扩展问题。

过渡句：当电路规模继续增加时，单机首先遇到的限制往往不是运行时间，而是内存容量。

第 4 页：单机证明生成的内存瓶颈

本页重点：解释为什么单机系统只能支持千万级电路。

演讲稿：

现实应用中的计算可能包含数千万甚至数十亿个门或约束。证明者需要保存多项式、拉格朗日基向量和群元素等大量中间数据，这些对象通常需要常驻内存。

论文指出，当时先进的单机系统大约只能处理一千万到两千万个门，再往上就会触发内存不足。其单位成本还超过每门 1 毫秒。

为了直观说明这个规模，论文以 SHA-256 为例。一次 SHA-256 压缩就需要超过 2.5 万个门。因此，一千万门大约只能完成 400 次压缩，对应可哈希的数据只有约 13 KB。对于真实的大规模应用，这个容量显然很有限。

所以这里的核心问题是：随着规模增加，单机物理内存先成为不可突破的硬边界。DIZK 因此选择把 Setup 和 Prover 分布到集群，让多台机器共同提供计算能力和内存容量。

过渡句： 基于这个瓶颈，论文提出了一个非常直接的研究问题。

第 5 页：研究问题与系统贡献

本页重点： DIZK 是系统层创新，不是新的密码学协议。

演讲稿：

DIZK 的研究问题是：能不能把 zkSNARK 的证明生成从一台机器搬到整个集群，并支持超大规模电路？

论文给出了两个标志性结果。第一，系统能够支持超过 10 亿条约束。第二，Prover 的平均单位成本约为每门 10 微秒。

它的贡献主要有三个方面。

第一，作者把 Groth 2016 zkSNARK 中的 Setup 和 Prover 改造成分布式任务，但保持 Verifier 不变。

第二，作者针对多项式运算、多标量乘和 QAP 归约设计了专门的分布式算子，重点减少全局 shuffle、内存放大和拖尾任务。

第三，作者在 Apache Spark 上用大约一万行 Java 实现了系统，并在十亿级约束和两个具体应用中进行了评测。

这里需要强调：DIZK 没有提出新的零知识证明协议。它的主要贡献是系统工程和分布式算法设计。

过渡句： 为了理解这些算法具体分布在哪里，先看证明生成的数据通路。

第 6 页：从 R1CS 到 QAP

本页重点： DIZK 分布式化的是约束归约、多项式运算和群运算。

演讲稿：

程序或计算首先会被编译成 R1CS，也就是一阶约束系统。它包含 (N) 个变量、(M) 个约束，以及三个稀疏矩阵 (a)、(b)、(c)。

随后，系统通过拉格朗日插值，把这些约束转换成 QAP，也就是二次算术程序。QAP 用只包含多项式的单一等式关系统一表达原来的全部约束，方便后续生成 zkSNARK 证明。

DIZK 不修改底层的配对密码学理论。它主要把 R1CS 到 QAP 的归约、后续多项式计算和群运算拆成可以在集群上执行的任务。

分布式 Setup 最终生成证明密钥和验证密钥，分布式 Prover 最终生成 128 字节的证明，而原有的单机 Verifier 可以直接使用。

过渡句： 下面这张架构图把 Setup、Prover 和 Verifier 的数据流完整地串联起来。

第 7 页：DIZK 系统架构

本页重点： 大对象分片存储，小证明回到普通验证者。

演讲稿：

这张图可以看作整篇论文的系统骨架。

在 Setup 通路中，系统执行 QAP 实例归约、拉格朗日求值和固定基多标量乘（将普通标量映射加密到椭圆曲线群上的点），也就是 fixMSM，最终生成分布式存储的证明密钥和很短的验证密钥。

在 Prover 通路中，系统执行 QAP 证人归约、FFT 和可变基多标量乘，也就是 varMSM，最后汇总得到 128 字节的证明。

R1CS 矩阵、证明密钥和证人等大对象，以 Spark RDD 的形式切分并存储在不同节点的内存中。公开输入规模较小，可以直接广播。

最重要的是，DIZK 汇聚了集群的 CPU 和内存，但没有改变验证接口。验证者仍然在单机上执行少量配对运算，输入验证密钥、公开输入和证明，输出接受或拒绝。

这种设计使 DIZK 生成的证明仍能兼容现有验证实现。

过渡句： 接下来分别看 DIZK 如何处理证明生成中的两类核心计算，首先是多项式运算。

第 8 页：分布式 FFT 与拉格朗日求值

本页重点： 两阶段 FFT 将多层通信压缩为一次全局 shuffle。

演讲稿：

多项式求值和插值是证明生成的主要工作之一。朴素算法的复杂度是 ($O(n^2)$)，无法处理大规模实例。串行 Cooley-Tukey FFT 可以把复杂度降低到 ($O(n \log n)$)，但如果直接搬到集群，每一层蝶形计算都需要重新排列整个数组。

在 Spark 中，这相当于每一层都进行一次全局 shuffle，通信成本非常高，而且会出现部分 executor 空转。

DIZK 采用两阶段 FFT。它把一个 (n) 点 FFT 拆成两批计算，每批包含大约 ($\sqrt{n}$) 个、规模为 ($\sqrt{n}$) 的 FFT。第一批在 mapper 端完成，然后进行一次全局 shuffle，第二批在 reducer 端完成。

这样，原本多层递归产生的多次全局通信，被压缩为一次明确的 shuffle。

Setup 侧还需要 Lag，也就是拉格朗日插值多项式的求值。当计算域是乘法子群时，这一步存在闭式解，各个 executor 可以独立计算自己的索引范围，不需要跨节点通信。

过渡句： 多项式运算之外，另一个大头是群上的多标量乘。

第 9 页：fixMSM 与 varMSM

本页重点： 固定基和可变基需要完全不同的数据布局。

演讲稿：

多标量乘可以理解为计算一组标量与群点乘积的总和。DIZK 根据基点是否固定，把它分为 fixMSM 和 varMSM。

在 fixMSM 中，基点是固定的，因此可以提前建立查找表。作者让 driver 统一预计算查找表，再广播到所有 executor。每个节点处理自己分到的标量，利用本地查表提高效率。这一设计用较小的广播成本换取了大量重复计算的减少。

在 varMSM 中，标量和基点都会变化。如果按标量桶进行全局分区，就需要重新 shuffle 大量群点，通信成本会非常高。DIZK 因此采用另一种布局：均匀切分输入对，让每个 executor 在本地运行 Pippenger 算法（解决可变基多标量乘的串行算法），最后只汇集各节点的局部群和，再进行一次归约相加。

这一页最值得记住的原则是：分布式化不能只是把串行循环机械地 map 出去，而要根据通信和数据移动成本重新安排算法。

过渡句：即使算法已经分布式化，稀疏矩阵中少数异常稠密的行列仍会造成严重的负载不均衡。

第 10 页：稀疏矩阵中的数据倾斜

本页重点：总体稀疏不代表每个分区都轻量。

演讲稿：

R1CS 矩阵总体上是稀疏的，但这种稀疏并不均匀。大多数行列只有少量非零元素，少数行或列却可能非常稠密。

在 Setup 侧，QAP 实例归约按列利用稀疏性。如果某个约束同时涉及成千上万个变量，对应的列就会变得非常稠密。负责这一列的 executor 可能直接耗尽内存。

在 Prover 侧，QAP 证人归约按行利用稀疏性。如果某个变量出现在大量约束中，对应的行会变得稠密。最典型的例子是常数 1，它可能参与几乎所有约束。负责该行的任务会远慢于其他任务，形成 straggler，也就是拖尾任务。

可以把它类比为食堂排队：大多数窗口只有几个人，但某个窗口排了几千人。虽然平均排队人数不高，整个流程仍会被最慢的窗口拖住。

通用的 skew join 或 block join 往往会复制数据、放大 RDD，并产生额外 shuffle，因此不能直接满足这里严格的内存要求。

过渡句：DIZK 的方法是先找出这些热点，再让稠密数据和稀疏数据走不同的处理路径。

第 11 页：负载均衡方案

本页重点：轻量识别热点，稠密列拆分，稀疏列保持常规处理。

演讲稿：

DIZK 的负载均衡方案分为两个部分。

第一部分是轻量统计。每个 executor 先统计自己分区中每一列的非零元素数量，再通过一次 shuffle 汇总。论文使用“非零元素数是否大于 $(\sqrt{M})$ ”作为启发式阈值，将列标记为稠密或稀疏。

这一步本身不会形成严重拖尾，因为每个分区只需要为每列提交一个统计值。

第二部分是双轨处理。绝大多数稀疏列继续使用常规 join 和 reduce，不进行额外复制。少数稠密列则被拆成多个分片，由不同 executor 并行计算，避免单个节点独自承担整列。

最后，系统把两部分结果 union 起来，再按行归约。

稠密行由约束系统的结构决定，与具体输入无关，因此 Setup 阶段识别一次以后，后续不同输入的 Prover 可以直接复用这一结果。

论文表明，通用 skew join 方案即使使用 128 个 executor，也无法处理 5000 万约束，并且速度约慢一倍。专门设计负载均衡对系统扩展非常关键。

过渡句： 在解读实验数字之前，还需要先明确论文所使用的软硬件环境。

第 12 页：实现与评测环境

本页重点： 所有绝对性能数字都依赖特定的 Spark 和 EC2 配置。

演讲稿：

DIZK 基于 Apache Spark 实现，核心逻辑使用 Java 编写，代码量大约一万行。

论文的单机基线是一台 AWS EC2 r3.large，配置为 2 个 vCPU 和 15 GiB 内存。分布式实验使用 r3.8xlarge 实例，每台有 32 个 vCPU 和 244 GiB 内存。10 台机器最多运行 128 个 executor，20 台机器最多运行 256 个 executor。

系统使用 256 位 Barreto-Naehrig 曲线。为了支持十亿级实例，作者还专门生成了满足 $(p-1)$ 能被 (2^{50}) 整除的曲线参数，以提供足够大的 FFT 域。

实验中的 instance size 指 R1CS 约束数 (M) ，变量数 (N) 与 (M) 大致相等。

论文没有重新评测 Verifier，因为验证端直接使用 libsnark 的实现。其耗时约为 2 毫秒，再加上每个公开输入域元素约 0.5 微秒。

因此，后面的结果应理解为在这套特定软硬件条件下得到的实验结论，换用不同环境时，绝对数值可能发生变化。

过渡句： 明确实验口径以后，先看 DIZK 最突出的结果，也就是它能够处理多大的实例。

第 13 页：可支持的实例规模

本页重点： 从单机千万级扩展到集群十亿级。

演讲稿：

这一页展示了 DIZK 最核心的结果。

图的横轴是 executor 数量，纵轴是以 2 为底的对数形式表示的约束规模。libsnark 的串行实现在大约一千万到两千万条约束时就因内存不足而退出。

DIZK 能够随着 executor 数量增加，持续扩大可处理的实例规模。在 256 个 executor 时，论文报告的最大规模达到约 (2^{31}) 条约束，也就是超过 20 亿。

论文把这一结果概括为：与当时先进的单机系统相比，DIZK 可支持的规模扩大了大约 100 倍。

过渡句： 能够跑更大的实例只是一个方面，下面继续看运行时间是否具有良好的扩展性。

第 14 页：时间与节点扩展性

本页重点： 随规模近线性增长，随节点增加接近线性加速。

演讲稿：

论文从弱扩展和强扩展两个方向评估系统。

弱扩展实验固定 executor 数量，逐步扩大实例规模。结果显示，Setup 和 Prover 的运行时间基本随约束规模近线性增长。这说明系统能够以相对稳定的单位成本处理更大的任务。

强扩展实验固定实例规模，再增加 executor 数量。运行时间总体上接近线性下降。当节点数量增加到一定程度以后，曲线会逐渐变平，因为任务调度、通信和结果归约等固定开销开始占据更大比例。

论文总结，DIZK 的 Prover 单位成本约为每门 10 微秒，而此前单机系统超过每门 1 毫秒，单位成本相差约 100 倍。

这里的“100 倍”指单位门成本的比较，并不意味着任何端到端应用都能稳定获得 100 倍加速。实际加速还取决于任务规模、集群配置和数据分布。

过渡句： 这些扩展结果并不是简单增加机器得到的，下一页展示专门算法优化带来的实际收益。

第 15 页：算法优化的效果

本页重点： 完整优化使可处理规模提高 10 倍，速度提高 2~4 倍。

演讲稿：

这一页评估前面介绍的算法重构是否真正有效。

上半部分分别展示 Lag、FFT、fixMSM 和 varMSM 的扩展表现。这些算子都呈现类似趋势：固定 executor 数量时，运行时间随规模近线性增长；固定实例规模时，增加 executor 可以明显降

低运行时间。

下半部分是系统级综合对照。论文使用 32 和 64 个 executor，对比完整优化版本和主要依赖通用分布式方案的未优化版本。

结果显示，完整优化版可处理的最大实例规模提高约 10 倍，Setup 和 Prover 的速度提高约 2 到 4 倍。

需要注意，论文比较的是整套优化组合与基线组合，并没有分别关闭每一个模块进行严格的孤立消融。因此，这些数字说明整体设计有效，但不能精确归因到某一个算法。

这组结果也表明，DIZK 的价值并不只是把任务放上 Spark。真正决定扩展性的，是对 FFT、多标量乘、稀疏归约和负载均衡的专门设计。

过渡句： 论文还用两个应用说明十亿级证明能力可以支持哪些实际场景，首先是照片可信编辑。

第 16 页：应用一——PhotoProof

本页重点： 在不公开原图的情况下，证明编辑操作符合规则。

演讲稿：

第一个应用是 PhotoProof，目标是提高新闻照片或刑侦照片的可信度。

相机在拍摄时，对原图的 commitment 进行硬件签名。之后，编辑者只能执行事先允许的操作，例如裁剪、旋转或模糊。发布编辑后的图片时，同时提供原图 commitment、签名和零知识证明。

第三方不需要获得原始图片，就可以验证当前图片确实来自一张经过签名的原图，并且只执行了允许的编辑操作。

论文在 2048 乘 2048 的黑白图片上进行了实验。裁剪约产生 420 万条约束，旋转约产生 1.38 亿条约束，模糊约产生 1360 万条约束。

表中的时间只表示在 64 个 executor 下生成约束和生成证人的时间，不包含 zkSNARK 的 Setup 和 Prover 时间。汇报时要特别区分这两个口径。

此外，论文实际实现的是黑白图片。RGB 彩色图片只作为可能的扩展方向，并没有在实验中完成。

过渡句： 第二个应用把类似的思想用于机器学习模型的训练完整性。

第 17 页：应用二——可验证机器学习

本页重点： 不公开训练数据，证明模型按公开算法正确训练。

演讲稿：

第二个应用关注机器学习模型的完整性。

设想医院掌握患者数据，这些数据受隐私和法律保护，不能直接公开。研究者使用这些数据训练并发布一个模型，但公众希望确认模型确实按照公开算法训练，没有被替换、投毒或伪造。

DIZK 的思路是把训练过程或检查关系编码成约束，并为正确执行生成零知识证明。这样，验证者可以检查模型训练的正确性，却不需要接触原始患者数据。

论文实现了线性回归、协方差矩阵和矩阵乘法。线性回归中并没有在电路内直接求逆，而是验证法方程：

$$\begin{bmatrix} X^T X w = X^T y \end{bmatrix}$$

这样可以把验证问题转化为更适合约束系统的矩阵乘法。

表中的时间仍然只表示约束生成和证人生成，不是完整的 Setup 与 Prover 时间。

还需要说明一个重要边界：论文没有评测输入数据与可信 commitment 是否一致的哈希证明。真实部署时必须补上这一环，否则系统只能证明“对某些私有数据正确执行”，不能证明这些数据就是预先承诺的真实数据。

过渡句： 最后，我们从部署成本和安全假设两个方面总结 DIZK 的局限。

第 18 页：边界、风险与总结

本页重点： DIZK 解决了可扩展性问题，但成本、可信设置和应用范围仍有限。

演讲稿：

DIZK 证明了 zkSNARK 的 Setup 和 Prover 可以扩展到集群，并将可处理规模从单机的千万级推进到十亿级。不过，这并不代表大规模证明已经变得廉价或容易部署。

第一，绝对成本仍然很高。即使平均成本降低到每门约 10 微秒，包含数十亿个门的电路仍需要让整个集群运行较长时间，计算、内存和电力成本都不可忽略。

第二，系统依赖电路特定的 trusted setup。只要电路发生变化，就需要重新生成参数。

第三，分布式环境增加了密钥安全难度。Setup 中产生的秘密随机数需要安全销毁。在数十台、多租户的计算节点上完成这件事，比保护单机内存更困难。论文没有解决分布式多方计算仪式的问题。

第四，应用评测仍然处于原型阶段。照片实验只覆盖黑白图像，机器学习实验只覆盖线性回归、协方差矩阵等相对简单的任务，不能直接外推到任意深度学习模型。

总体来看，DIZK 的意义是证明了一个重要方向：零知识证明的生成过程可以通过专门的分布式算法突破单机内存边界，同时保留短证明和轻量验证的优势。它为后续大规模零知识证明系统提供了系统设计和工程实现上的参考。

我的汇报到这里结束，谢谢各位老师和同学，欢迎提问。

汇报时可重点记忆的数字

- 论文：DIZK, USENIX Security 2018。
- 证明大小：128 B。
- 单机上限：约 1000 万~2000 万门或约束。
- DIZK 最大规模：超过 10 亿条约束，实验图中约达到 (2^{31})。
- Prover 单位成本：约 10 微秒每门。
- 实现：Apache Spark, 约一万行 Java。
- 集群：最多 256 个 executor。
- 优化效果：可处理规模约提高 10 倍，Setup 和 Prover 约加速 2~4 倍。

可能被问到的问题

1. DIZK 是否提出了新的零知识证明协议？

没有。DIZK 主要是系统层工作。它把已有 zkSNARK 协议中的 Setup 和 Prover 分布式化，并针对 Spark 环境重新设计关键算子。

2. 为什么 Verifier 不需要分布式化？

因为 zkSNARK 的证明是常数大小，验证只需要少量配对运算，成本与大规模电路相比很低。DIZK 的瓶颈集中在 Setup 和 Prover。

3. 为什么不能直接把串行算法放到 Spark 上？

直接迁移会产生大量全局 shuffle、数据复制和拖尾任务。DIZK 根据通信成本重新设计了 FFT、多标量乘和稀疏矩阵归约。

4. DIZK 最大的安全风险是什么？

它依赖可信设置。分布式 Setup 需要在多台机器上保护并销毁秘密随机数，攻击面比单机更大，而论文没有给出完整的分布式 MPC 仪式。

5. 论文中的应用时间是否等于生成零知识证明的总时间？

不等于。应用表格中的秒数主要是约束生成和证人生成时间，不包含完整的 zkSNARK Setup 和 Prover 时间。